ethercat主站接口文档-slowly

1. 系统启动流程

1.1电脑连接Orin

- 通过网线将开发调试电脑与机器人背部网口接口连接；
- 机器人上电，Orin运行，Orin外置网口默认IP为172.16.0.10；
- 将调试电脑的网络配置为与Orin同一网段，例如设置固定Ip为172.16.0.100；
- 在调试电脑ping通Orin，即可通过ssh nvidia@172.16.0.10连接Orin，密码为123456

1.2 EtherCAT 中间件启动

代码块

```
# 1. 配置EtherCAT
/workspace/embed_fusion/bin/run_ethercat_station.bash
# 2. 监控从站状态（在新终端执行）
watch -t 1 ethercat slaves
```

监控状态说明：

- 理想状态：所有从站进入 `PRE-OP` 状态
- 异常处理：如果出现错误状态，需重启电机设备

2、通讯接口设计

2.1 共享内存通信机制

- 通信频率：1000Hz (1ms周期)
- 实现原理：通过共享内存实现高速数据交换
- 通信结构体：
  - `CommandSharedMemory`：应用层→EtherCAT主站
  - `StatusSharedMemory`：EtherCAT主站→应用层

2.2 数据结构定义

2.2.1 控制指令结构

代码块

```
const int MAX_JOINTS = 35;

// 共享内存-控制指令
struct CommandSharedMemory {
    uint64_t frame_counter;           // 帧计数器
    uint8_t joint_count;              // 电机数量
    uint16_t hand_count;              // 手部数量

    JointCmdData joint_commands[MAX_JOINTS]; // 腿部/手臂关节控制指令
    HandCommand hand_commands[2];     // 手部控制指令 (0:左,1:右)
    uint64_t timestamp;                // 时间戳
    uint32_t crc;                      // CRC32校验
};

// 单个电机控制指令
struct __attribute__((__packed__)) JointCmdData {
    uint16_t Controlword;     // 控制字
    int32_t pos_target;       // 目标位置
    int32_t vel_target;       // 目标速度
    int16_t torque_target;    // 目标扭矩
    int16_t mode_operation;   // 操作模式
    int16_t kp;               // P增益
    int16_t kd;               // D增益
};

// 手部控制指令
struct HandCommand {
    uint16_t control_word;    // 控制字
    uint8_t hand_id;          // 手ID (0:左,1:右)
    uint8_t reserved;         // 保留字段
    HandJointCommand joints[6]; // 6个关节指令
};

// 手部关节控制指令
struct HandJointCommand {
    int16_t target;           // 目标位置
    int16_t param;            // 速度/时间参数
    HandControlMode mode;     // 控制模式
    uint8_t options;          // 预留选项
};
```

2.2.2 状态反馈结构

代码块

```
// 共享内存-状态反馈
struct StatusSharedMemory {
    uint64_t frame_counter;          // 帧计数器
    uint8_t joint_count;              // 电机数量
    uint16_t hand_count;              // 手部数量

    JointStatusData joint_status[MAX_JOINTS]; // 关节状态反馈
    HandStatus hand_status[2];        // 手部状态反馈 (0:左,1:右)
    uint64_t timestamp;               // 时间戳
    uint32_t crc;                     // CRC32校验
};

// 单个电机状态反馈
struct __attribute__ ((__packed__)) JointStatusData {
    uint16_t Statusword;      // 状态字
    int32_t pos_real;         // 当前位置
    int32_t vel_real;         // 当前速度
    int16_t torque_real;      // 当前扭矩
    int16_t mode_operation;   // 当前操作模式
    uint16_t error_code;      // 错误码
};

// 手部整体状态
struct HandStatus {
    uint16_t global_status;     // 全局状态字
    uint16_t error_code;        // 错误代码
    HandJointStatus joints[6];  // 6个关节状态
};

// 手部关节状态
struct HandJointStatus {
    int16_t position;           // 位置 (0.1度)
    int16_t velocity;           // 速度 (0.1°/s)
    int16_t current;            // 电流 (mA)
    uint16_t status;            // 状态字
};
```

2.3 操作模式说明

支持的电机控制模式：

- MIT模式（`mode_operation = -1`）

- 其他模式
  - 模式切换时必须先执行电机失能操作

代码块

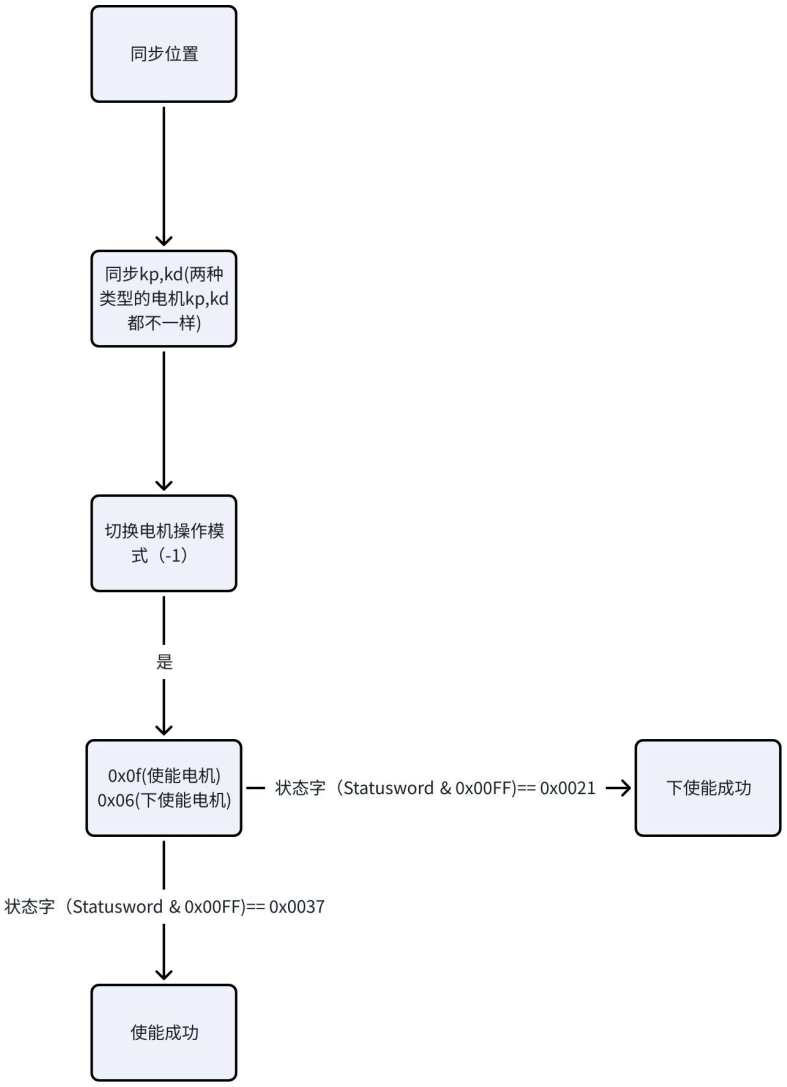
```
// MIT模式配置示例
JointCmdData motorCmd = {
    .Controlword = 0xf,  //使能电机
    .pos_target = targetPosition,
    .vel_target = targetVelocity,
    .torque_target = targetTorque,
    .mode_operation = -1,  // PD阻抗模式
    .kp = 500,             // P增益系数
    .kd = 20               // D增益系数
};
```

2.4 接口函数说明

2.4.1 ShareMemory 类

1	*ShareMemory：模板类构造函数*	ShareMemory<CommandSharedMemory, StatusSharedMemory> *shm = new ShareMemory<CommandSharedMemory, StatusSharedMemory>("shm_mc"); • "shm_mc"：指定shm的命名，这些参数都是要和主站沟通好，目前不能改动，否则无法通讯。
2	*以下都是ShareMemory的方法：* *void* start()	开始启动通讯
3	*void* resetQueueZero()	清空队列
4	*bool* sendData(const *SEND_ELEM* **in_data*)	发送数据，参数:模板类型的指针
5	*bool* receiveData(*RECV_ELEM* **out_data*)	接收数据，参数:模板类型的指针

3、电机启动与参数设置



3.1电机顺序索引映射表

JointCmdData joint_commands[35];里面的映射关系：

身体部位	索引范围	有效关节数	有效索引	关节类
左腿	0-5	6	0,1,2,3,4,5	髋Pitch→髋Roll→髋Ya
右腿	6-11	6	6,7,8,9,10,11	髋Pitch→髋Roll→髋Ya
左臂	12-18	7	12,13,14,15,16,17,18	肩Pitch→肩Roll→肩Yaw→肘→
右臂	19-25	7	19,20,21,22,23,24,25	肩Pitch→肩Roll→肩Yaw→肘→
头部	26-27	2	26,27	颈部Yaw→
腰部	28	1	28	腰部
预留	29-34	0	-	备用

3.2各个关节的 PD 参数

身体部位	关节索引	关节类型	关节参数			
			P增益(kp)	参考值	D增益(kd)	参考值
左腿	0	髋Pitch	0~500	400	0~50	20
	1	髋Roll	0~500	500	0~50	10
	2	髋Yaw	0~500	380	0~5	5
	3	膝关节	0~500	440	0~50	17
	4	踝Pitch	0~500	120	0~5	3
	5	踝Roll	0~500	80	0~5	3
右腿	6	髋Pitch	0~500	400	0~50	20
	7	髋Roll	0~500	500	0~50	10
	8	髋Yaw	0~500	380	0~5	5
	9	膝关节	0~500	440	0~50	17
	10	踝Pitch	0~500	120	0~5	3
	11	踝Roll	0~500	80	0~5	3
左臂	12	肩Pitch	0~500	150	0~5	5
	13	肩Roll	0~500	150	0~5	5
	14	肩Yaw	0~500	150	0~5	5
	15	肘关节	0~500	100	0~5	5
	16	腕部yaw	0~500	100	0~5	5
	17	腕部 Pitch	0~500	80	0~5	4
	18	腕部 Roll	0~500	80	0~5	4
右臂	19	肩Pitch	0~500	150	0~5	5
	20	肩Roll	0~500	150	0~5	5
	21	肩Yaw	0~500	150	0~5	5
	22	肘关节	0~500	100	0~5	5
	23	腕部yaw	0~500	100	0~5	5
	24	腕部 Pitch	0~500	80	0~5	4
	25	腕部 Roll	0~500	80	0~5	4
头部	26	pitch	0~500	30	0~5	2
	27	yaw	0~500	30	0~5	2
腰部	28	yaw	0~500	500	0~5	5

4、注意事项

1. 共享内存ID不可更改
   - 必须使用 `"shm_mc"` 作为共享内存命名
2. 模式切换要求
   - 每次切换操作模式前必须先失能电机
   - 否则可能导致系统异常或设备损坏
3. 控制指令安全
   - 避免发送超出合理范围的指令值
   - 当 `error_code != 0` 时应立即进入安全处理流程
4. 实时性保证
   - 控制循环必须保证1ms周期
   - 长时间未收到状态反馈时应进入安全状态